

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Centro Universitario de Occidente

División de Ciencias de la Ingeniería

Introducción a la Programación y Computación 2

Ing. Daniel González | Auxiliar: Roberto Bautista

MANUAL TÉCNICO

ConnectWork

Plataforma Freelance de Servicios Digitales

Carlos Enrique Yacabalquiej Sacalxot 202230581

1. Arquitectura del sistema

ConnectWork utiliza una arquitectura cliente-servidor separada, con un backend REST API y un frontend SPA (Single Page Application).

1.1 Stack tecnológico

Backend

Componente	Tecnología
Lenguaje	Java 17
Framework	Jakarta EE 10 (Servlets puros)
Servidor	Apache Tomcat 10.1.24
Base de datos	MySQL 8 (vía XAMPP)
Build	Maven
Autenticación	JWT (jjwt 0.12.5)
Encriptación	BCrypt (bcrypt 0.4)
JSON	Gson 2.10.1
Conector BD	mysql-connector-j 8.3.0

Frontend

Componente	Tecnología
Framework	Angular 21 (Standalone Components)
Lenguaje	TypeScript (estricto)
Estado reactivo	Signals
Íconos	Heroicons (SVG inline)
Estilos	CSS Variables + CSS puro
Exportación PDF	jsPDF + jspdf-autotable

Herramientas de desarrollo

Herramienta	Uso
NetBeans IDE	Desarrollo del backend Java
VS Code	Desarrollo del frontend Angular

Git / GitHub	Control de versiones
Thunder Client	Pruebas de API REST
XAMPP	Servidor MySQL local
Brave Browser	Navegador para prueba

1.2 Patrón de arquitectura

El sistema sigue el patrón de arquitectura MVC (Modelo-Vista-Controlador) adaptado a una aplicación web REST:

- Modelo: Clases Java (POJOs) en el paquete `com.connectwork.model`
- Controlador: Servlets en `com.connectwork.servlets` que procesan las peticiones HTTP
- Acceso a datos: DAOs en `com.connectwork.dao` que encapsulan las consultas SQL
- Vista: Componentes Angular que consumen la API REST
- Utilidades: Clases auxiliares en `com.connectwork.util` (conexión BD, JWT, BCrypt, Gson, CORS)

3.3 Flujo de comunicación

El frontend Angular se comunica con el backend mediante peticiones HTTP (GET, POST, PUT) a endpoints REST. El backend responde en formato JSON. La autenticación se maneja mediante tokens JWT enviados en el header `Authorization` de cada petición.

Flujo típico: Usuario interactúa con componente Angular → Servicio Angular envía petición HTTP con token JWT → JWTFilter valida el token → Servlet procesa la petición → DAO ejecuta consulta SQL → Respuesta JSON al frontend

2. Base de datos

2.1 Configuración

Motor: MySQL 8 (vía XAMPP)

Host: localhost:3306

Schema: connectwork

Usuario: root

Contraseña: (vacía)

2.2 Tablas del sistema

El sistema cuenta con las siguientes tablas:

Tabla: usuarios

Almacena la información de todos los usuarios del sistema (clientes, freelancers y administradores).

Columna	Tipo	Nulo	Descripción
id_usuario	INT (PK, AI)	NO	Identificador único

nombre_completo	VARCHAR(150)	NO	Nombre del usuario
username	VARCHAR(50) UNIQUE	NO	Nombre de usuario
email	VARCHAR(100)	NO	Correo electrónico
password	VARCHAR(255)	NO	Contraseña (BCrypt)
cui	VARCHAR(20)	SÍ	CUI del usuario
telefono	VARCHAR(20)	SÍ	Teléfono
rol	ENUM	NO	CLIENTE, FREELANCER, ADMINISTRADOR
saldo	DECIMAL(10,2)	NO	Saldo en la plataforma
activo	BOOLEAN	NO	Estado de la cuenta
primera_vez	BOOLEAN	NO	Requiere completar info

Tabla: proyectos

Almacena los proyectos publicados por los clientes.

Columna	Tipo	Nulo	Descripción
id_proyecto	INT (PK, AI)	NO	Identificador único
id_cliente	INT (FK)	NO	Cliente que publicó
id_categoria	INT (FK)	NO	Categoría del servicio
titulo	VARCHAR(200)	NO	Título del proyecto
descripcion	TEXT	NO	Descripción detallada
presupuesto	DECIMAL(10,2)	NO	Presupuesto máximo
fecha_limite	DATE	SÍ	Fecha límite de entrega
estado	ENUM	NO	Estado del proyecto
fecha_creacion	DATETIME	NO	Fecha de publicación

Tabla: propuestas

Almacena las propuestas enviadas por freelancers a los proyectos.

Tabla: contratos

Se genera automáticamente al aceptar una propuesta. Contiene el monto bloqueado, porcentaje de comisión vigente al momento de la firma, y el estado del contrato (ACTIVO, COMPLETADO, CANCELADO).

Tabla: entregas

Registra las entregas realizadas por el freelancer dentro de un contrato. Incluye la descripción del trabajo, URL de archivos adjuntos, estado (PENDIENTE, APROBADA, RECHAZADA) y motivo de rechazo si aplica.

Tabla: calificaciones

Almacena las calificaciones (1 a 5 estrellas) y comentarios que el cliente otorga al freelancer al completar un contrato.

Tabla: categorias

Catálogo de categorías de servicios (Diseño Gráfico, Desarrollo Web, Marketing, etc.). Pueden activarse o desactivarse.

Tabla: habilidades

Catálogo de habilidades asociadas a cada categoría. Los freelancers seleccionan sus habilidades al completar su perfil.

Tabla: recargas

Historial de recargas de saldo realizadas por los clientes.

Tabla: configuracion_comision

Almacena la configuración vigente del porcentaje de comisión de la plataforma.

Tabla: historial_comisiones

Registro histórico de los cambios de porcentaje de comisión.

Tabla: saldo_plataforma

Saldo global acumulado de comisiones de la plataforma.

Tablas: solicitudes_categoria y solicitudes_habilidad

Almacenan las solicitudes de nuevas categorías (por clientes) y habilidades (por freelancers) que deben ser aprobadas o rechazadas por el administrador.

Tablas intermedias

proyecto_habilidades: relación muchos a muchos entre proyectos y habilidades requeridas.
freelancer_habilidades: relación muchos a muchos entre freelancers y sus habilidades.

3. Estructura del backend

3.1 Paquetes Java

Paquete	Descripción
---------	-------------

com.connectwork.model	Clases POJO: Usuario, Proyecto, Propuesta, Calificacion, Categoria, Habilidad, Comision
com.connectwork.dao	Acceso a datos: UsuarioDAO, ProyectoDAO, PropuestaDAO, EntregaDAO, CalificacionDAO, CategoriaDAO, HabilidadDAO, ComisionDAO, RecargaDAO, ReporteDAO, SolicitudDAO, CargaDatosDAO, ClienteDAO, FreelancerDAO
com.connectwork.servlets	Controladores REST: AuthServlet, UsuarioServlet, ProyectoServlet, PropuestaServlet, EntregaServlet, CalificacionServlet, CategoriaServlet, HabilidadServlet, ComisionServlet, SaldoServlet, ReporteServlet, SolicitudServlet
com.connectwork.util	Utilidades: ConexionBD, JWTUtil, JWTFilter, PasswordUtil (BCrypt), GsonUtil, CorsFilter

3.2 Endpoints REST principales

Método	Ruta	Descripción
POST	/auth/login	Inicio de sesión, retorna JWT
POST	/auth/registro	Registro de nuevo usuario
GET	/proyectos	Listar proyectos
POST	/proyectos	Crear proyecto (cliente)
POST	/propuestas	Enviar propuesta (freelancer)
PUT	/propuestas/{id}/aceptar	Aceptar propuesta (cliente)
POST	/entregas	Subir entrega (freelancer)
PUT	/entregas/{id}/aprobar	Aprobar entrega (cliente)
POST	/calificaciones	Calificar freelancer (cliente)
GET	/reportes/*	Reportes por rol

3.3 Seguridad

La autenticación se implementa mediante JSON Web Tokens (JWT). Al iniciar sesión, el servidor genera un token que contiene el ID del usuario, su rol y nombre de usuario. Este token se envía en cada petición posterior en el header Authorization: Bearer <token>.

El JWTFilter intercepta todas las rutas protegidas, valida el token, y extrae los datos del usuario para que los servlets puedan acceder a ellos. Las contraseñas se almacenan encriptadas con BCrypt.

3.4 Manejo de transacciones

Las operaciones críticas del sistema utilizan transacciones SQL con setAutoCommit(false), commit y rollback para garantizar la integridad de los datos. Esto incluye: aceptar propuestas (7 pasos), aprobar entregas (5 pasos), y cancelar contratos (3 pasos).

4. Estructura del frontend

4.1 Componentes principales

Módulo Admin	Módulo Cliente	Módulo Freelancer
dashboard-admin	dashboard-cliente	dashboard-freelancer
gestion-categorias	publicar-proyecto	explorar-proyectos
gestion-habilidades	mis-proyectos	mis-propuestas
gestion-usuarios	ver-propuestas	contratos-activos
cambiar-comision	recargar-saldo	subir-entrega
solicitudes-pendientes	revisar-entrega	reportes-freelancer
reportes-admin	calificar-freelancer	
	reportes-cliente	

4.2 Servicios Angular

Cada módulo funcional cuenta con un servicio inyectable que encapsula las llamadas HTTP al backend: auth.service, categoria.service, habilidad.service, comision.service, proyecto.service, propuesta.service, entrega.service, calificacion.service, saldo.service, reporte.service, solicitud.service.

4.3 Guards de ruta

El sistema implementa dos guards: authGuard (verifica que el usuario tenga sesión activa) y roleGuard (verifica que el rol del usuario coincida con los roles permitidos para la ruta).

4.4 Interceptor HTTP

Un interceptor agrega automáticamente el token JWT a todas las peticiones HTTP salientes, evitando tener que hacerlo manualmente en cada servicio.

5. Instrucciones de instalación y despliegue

5.1 Requisitos previos

- Java JDK 17 o superior
- Apache Tomcat 10.1.24
- MySQL 8 (vía XAMPP recomendado)
- Node.js (versión LTS) y npm
- Angular CLI
- Maven
- Git

5.2 Base de datos

1. Iniciar el servicio MySQL desde XAMPP.
2. Crear la base de datos ejecutando el script SQL proporcionado en el repositorio.
3. La base de datos se crea con el nombre 'connectwork' en localhost:3306 con usuario root y sin contraseña.

5.3 Backend

1. Abrir el proyecto en NetBeans IDE.
2. Ejecutar Clean and Build para compilar el proyecto Maven.
3. Desplegar el archivo WAR generado en Apache Tomcat 10.1.24.
4. El backend estará disponible en: <http://localhost:8080/connectwork-backend>

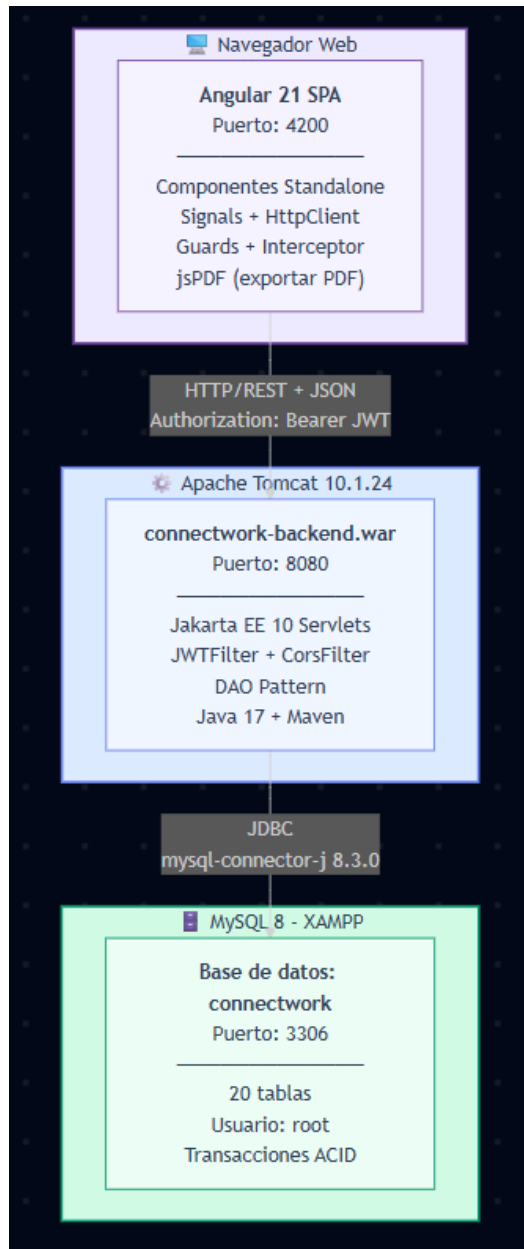
5.4 Frontend

1. Abrir una terminal en la carpeta Frontend del proyecto.
2. Ejecutar 'npm install' para instalar las dependencias.
3. Ejecutar 'ng serve' para iniciar el servidor de desarrollo.
4. El frontend estará disponible en: <http://localhost:4200>

5.5 Usuarios de prueba

Usuario	Contraseña	Rol	Notas
admin	admin123	ADMINISTRADOR	Precargado en BD
maria_cliente	password123	CLIENTE	Info completada
juan.dev	password123	FREELANCER	Info completada

6. Diagrama de despliegue



El sistema se despliega en un entorno local con los siguientes nodos:

- Navegador web (Brave/Chrome): ejecuta la aplicación Angular (SPA) en el puerto 4200.
- Servidor Tomcat 10.1.24: aloja el archivo WAR del backend en el puerto 8080.
- Servidor MySQL 8 (XAMPP): base de datos connectwork en el puerto 3306.

La comunicación entre el frontend y el backend se realiza mediante HTTP/REST con JSON. La comunicación entre el backend y la base de datos se realiza mediante JDBC.

7. Consideraciones adicionales

7.1 Variables CSS globales

El sistema utiliza un sistema de diseño basado en CSS Variables para mantener consistencia visual: --primario: #667eea, --secundario: #764ba2, --texto: #2E4057, --texto-suave: #6c757d, --borde: #e1e5eb, --fondo: #f7f8fa.

7.2 Manejo de errores

El sistema implementa validaciones dobles: en el frontend (formularios con validaciones reactivas) y en el backend (validaciones en cada servlet antes de procesar). Los errores se comunican al usuario mediante mensajes descriptivos que aparecen y desaparecen automáticamente después de 3 segundos.

7.3 Repositorio

GitHub: <https://github.com/CarlosGots>

El código fuente completo del proyecto se encuentra en el repositorio de GitHub del estudiante.